Similarity and distances

Irene Siragusa, PhD

CHILAB
Università degli Studi di Palermo

Outline

- Introduction
- Multidimensional data
- Textual data
- Binary and set data
- Temporal series
- Graphs

Introduction

Given two objects O_1 and O_2 we can determine:

- A value of distance $Dist(O_1, O_2)$, where smaller values imply a greater similarity.
- A value of similarity $Sim(O_1, O_2)$ where larger values imply a greater similarity.

- Both distances and similarity functions can be expressed:
  - In a closed form
  - Defined algorithmically

Multidimentional Data

- Quantitative data
  - L_p-norm
  - Minkowski distance
  - Proximity thresholding
  - Mahalanobis distance
  - Geodesic distances (ISOMAP)
- Categorical data
  - Inverse occurrence frequency
  - Goodall similarity
- Mixed quantitative and categorical data
  - Weighted average
  - Normalized weighted average

L_p-norm

- Given two data points $\bar{X} = (x_1 \dots x_d)$ and $\bar{Y} = (y_1 \dots y_d)$ the L_p-norm can be defined a

$$Dist(\bar{X}, \bar{Y}) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

L_p-norm

- $p = 1$ Manhattan distance
  - Sum of absolute values
- $p = 2$ Euclidean distance
  - Square root of sum of squares
  - Rotation invariant
- $p = \infty$ Infinity norm or Chebyshev distance
  - Largest absolute value

Minkowski distance

- Generalize version of the L_p-norm

- Weight the relative importance of different features relying on domain-knowledge

$$Dist(\bar{X}, \bar{Y}) = \left(\sum_{i=1}^{d} a_i \cdot |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Curse of dimensionality

- Which value of *p*? The larger the best?

- Many distance-based data mining applications lose their effectiveness as the dimensionality of the data increases.

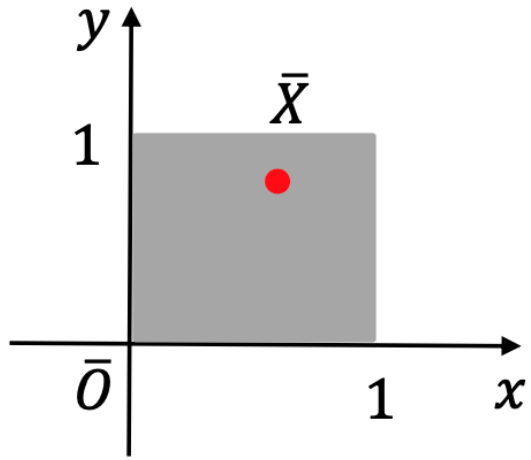
  - e.g. a distance-based clustering algorithm may group unrelated data points because the distance function may poorly reflect the intrinsic semantic distances between data points with increasing dimensionality.

Curse of dimensionality

- Consider a unit cube of dimensionality *d* in the nonnegative quadrant, in which $\bar{X}$ is a random point in the cube, and calculate the Manhattan distance between $\bar{O}$ and $\bar{X}$, whose components can be seen as X_i random variables in [0,1]

$$Dist(\bar{O}, \bar{X}) = \sum_{i=1}^{d} (X_i - 0)$$

- $Dist(\bar{O}, \bar{X})$ results a random variable with
  - $\mu = d/2$
  - $\sigma = \sqrt{d/12}$



Curse of dimensionality

As *d* increases, by the law of large numbers, the vast majority of randomly chosen points inside the cube will lie in the range:

$$[D_{min}, D_{max}] = [\mu - 3\sigma, \mu + 3\sigma]$$

the distance from the origin will lie in a range:

$$D_{max} - D_{min} = 6\sigma = \sqrt{3d}$$

that depends on *d*, thus the expected Manhattan distance grows with dimensionality at a rate that is linearly proportional to *d*.

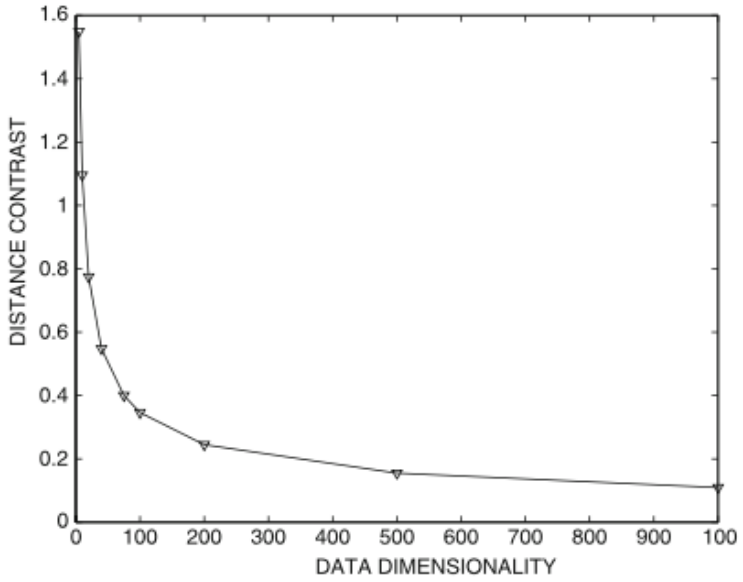
Curse of dimensionality

We can calculate the contrast, the ratio of the variation in the distances to the absolute values

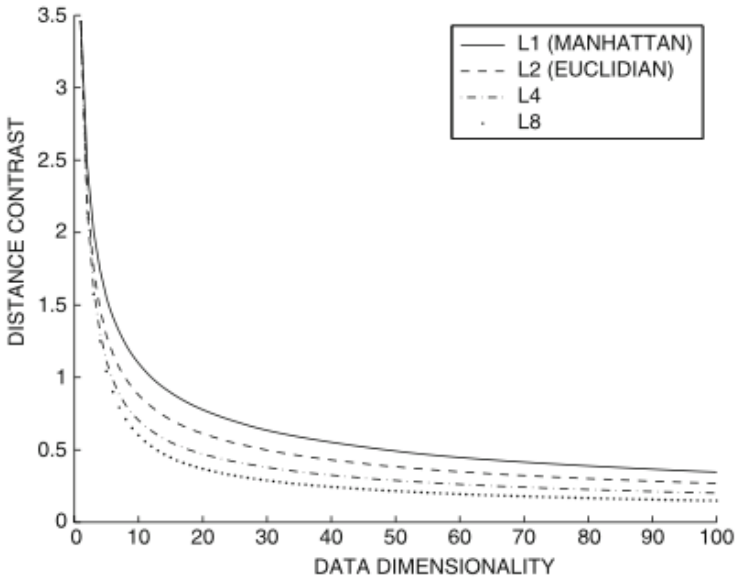
$$\text{Constrast}(d) = \frac{D_{max} - D_{min}}{\mu} = \sqrt{12/d}$$

This value can be seen as the relative dimension of the measured value with respect to the corresponding mean values

Curse of dimensionality



(a) Contrasts with dimensionality



(b) Contrasts with norms

- Contrast is decreasing as *d* increases
- As *p* increases, the decreasing trend in contrast is larger

Curse of dimensionality

- L_p-norm suffer from the additive noise effects of the irrelevant features

- The larger the dimensionality, the lower the value of *p*

  - dimensionality is not the only factor in the choice of *p*.

  - *p* should be selected in an application-oriented way.

Curse of dimensionality

- In high-dimensional data, not all the features are relevant

  - ✅ dropping irrelevant features
  - ✅ reduce noise in data, since small variations in high-dimensions can hide similarities
  - ✅ apply data reduction and data transformation algorithms
  - ✅ consider the appropriate p value, or use a fractional-norm, an L_p-norm with $p \in (0,1)$

Proximity thresholding

- Method to de-emphasize levels of dissimilarity in a dimensionality-sensitive way.
  - Discretized each feature into k_d equi-depth buckets
  - Given $\bar{X} = (x_1 \dots x_d)$ and $\bar{Y} = (y_1 \dots y_d)$ two *d*-dimensional records, if both x_i and y_i belong to the same bucket, the two records are in proximity on dimension *i*.
  - A proximity set $S(\bar{X}, \bar{Y}, k_d)$ can be created considering the dimensions subset in which $\bar{X}$ and $\bar{Y}$ map to the same bucket

Proximity thresholding

- For each dimension, considering the m_i and n_i as the maximun and minimum value in of the bucket in *i*-th dimension, the $PSelect(\bar{X}, \bar{Y}, k_d)$ similarity can be calculated as

$$PSelect(\bar{X}, \bar{Y}, k_d) = \left[\sum_{i \in S(\bar{X}, \bar{Y}, k_d)} \left(1 - \frac{|x_i - y_i|}{m_i - n_i} \right)^p \right]^{1/p}$$

- It ranges from 0 to $|S(\bar{X}, \bar{Y}, k_d)|$ since each calculated element ranges in [0,1]

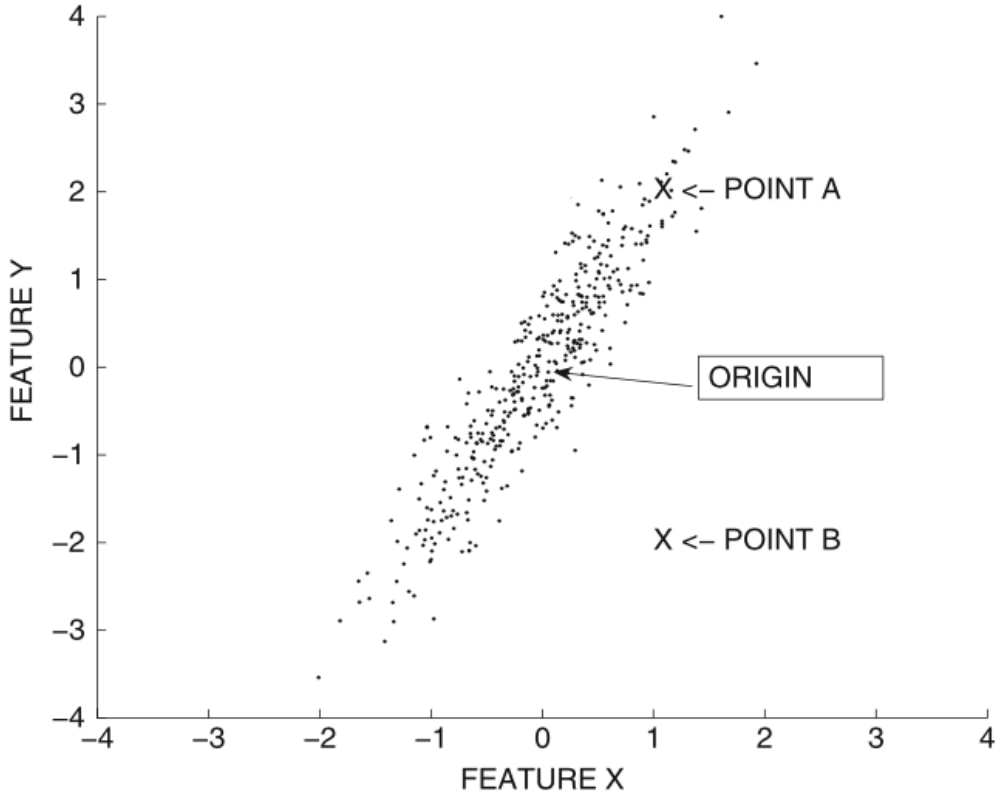
Proximity thresholding

- This is a similarity function since larger values imply greater similarity.

- A nonzero similarity component is only for dimensions mapping to the same bucket. Similar records, will have a larger number of similarity components, and each will contribute more to the similarity value.

Impact of data distribution

A and B are equi-distant from the origin?

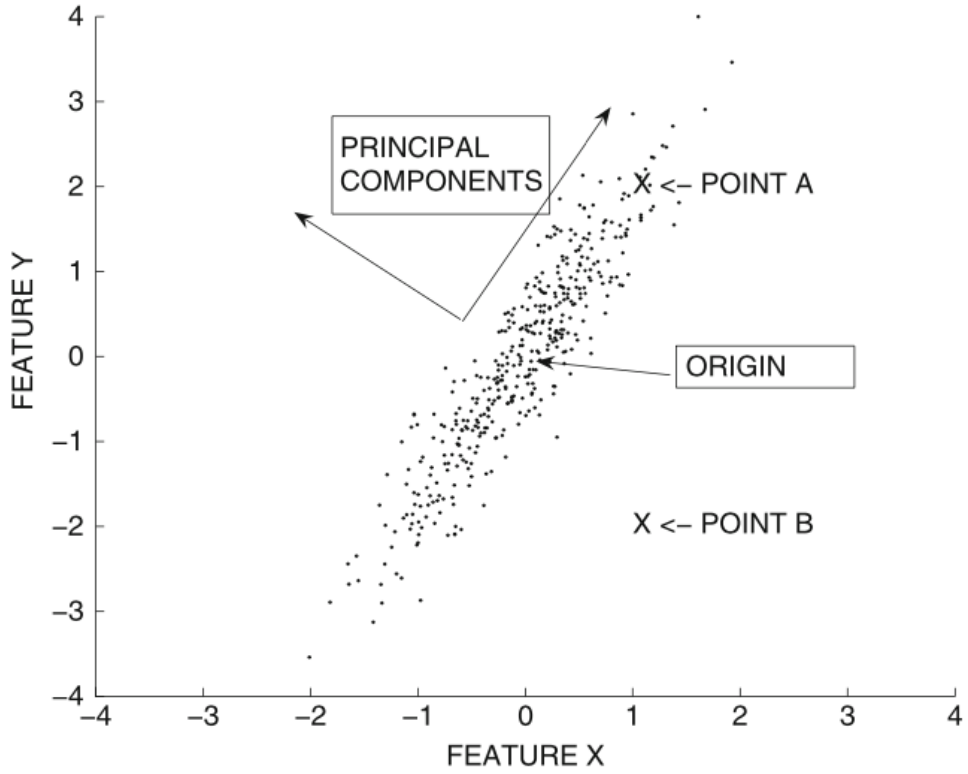


Impact of data distribution

- A and B are equi-distant from the origin?

☑ L_p-norm

☒ Variance
  - OA is aligned with a high-variance direction
  - OB is aligned with a low-variance direction
  - OA ought to be less than OB



Mahalanobis distance

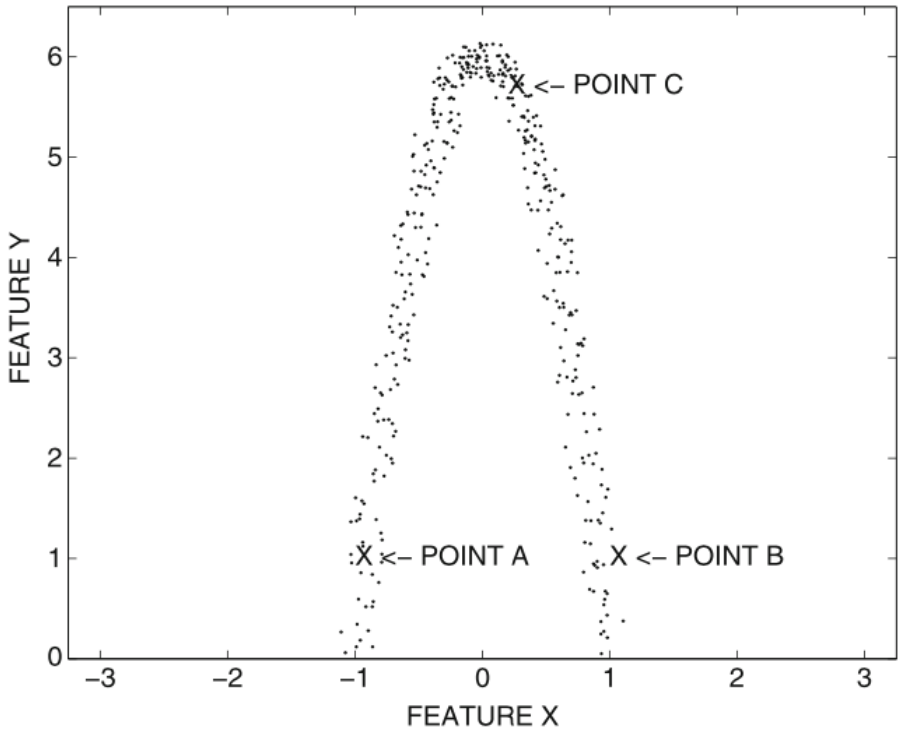
- Let Σ be the $d \times d$ covariance matrix of the data set. The Mahalanobis distance between two d-dimensional data points $\bar{X}$ and $\bar{Y}$ is defined as

$$Maha(\bar{X}, \bar{Y}) = \sqrt{(\bar{X} - \bar{Y})\Sigma^{-1}(\bar{X} - \bar{Y})^T}$$

- It leverages data's distribution
- Mahalanobis distance is equivalent to the Euclidean distance after PCA

Nonlinear distributions

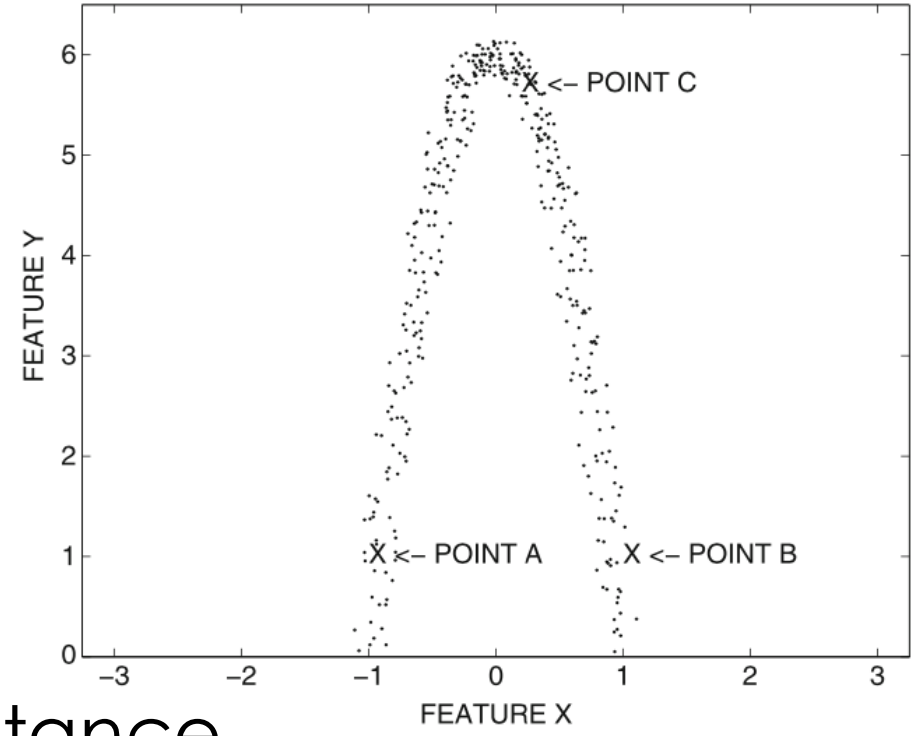
Is B or C closer to A?



Nonlinear distributions

Is B or C closer to A?

- Data distribution suggest something different with respect to the Euclidian distance
- We can consider the shortest length of the path from one data point to another, with point-to-point jumps from data points to one of their k-nearest neighbors based on Euclidean distance



Geodesic distances

- The overall sum of the point-to-point jumps reflects the aggregate change (distance) from one point to another (distant) point more accurately than a straight-line distance between the points.

- The implicit assumption is that nonlinear distributions are locally Euclidean but are globally far from Euclidean.

ISOMAP

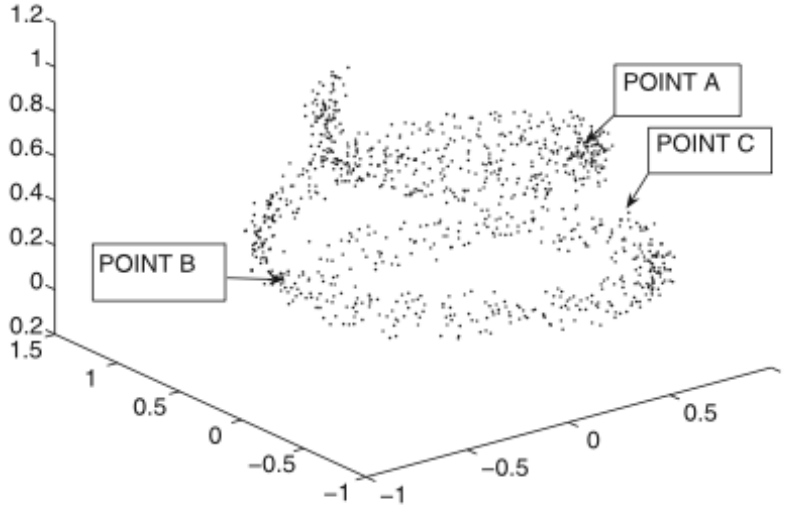
- Such geodesic distances can be computed via ISOMAP, approach derived from a nonlinear dimensionality reduction and embedding method

ISOMAP

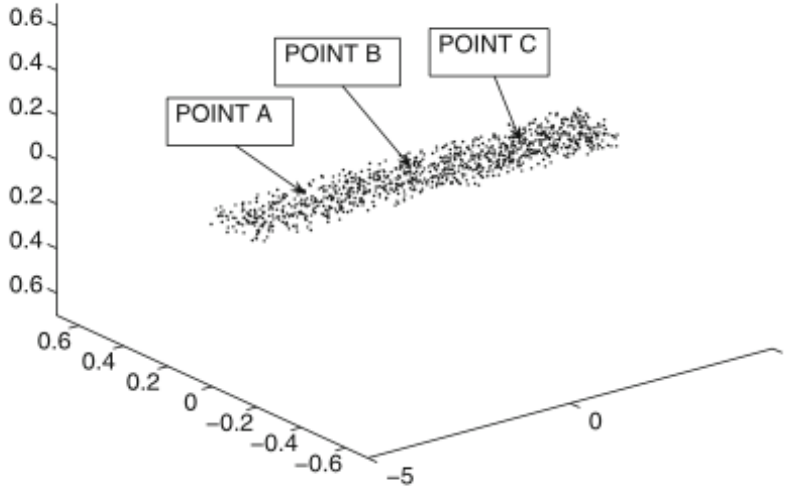
- Compute the *k*-nearest neighbors of each point
- Construct a weighted graph G
  - nodes -> data points
  - edge weights -> Euclidean distance of these *k*-nearest neighbors
- For any pair of points $\bar{X}$ and $\bar{Y}$, report $Dist(\bar{X}, \bar{Y})$ as the shortest path between the corresponding nodes in G
- Dimensionality reduction via MDS

ISOMAP

- The overall effect of the approach is to *straighten out* the nonlinear shape



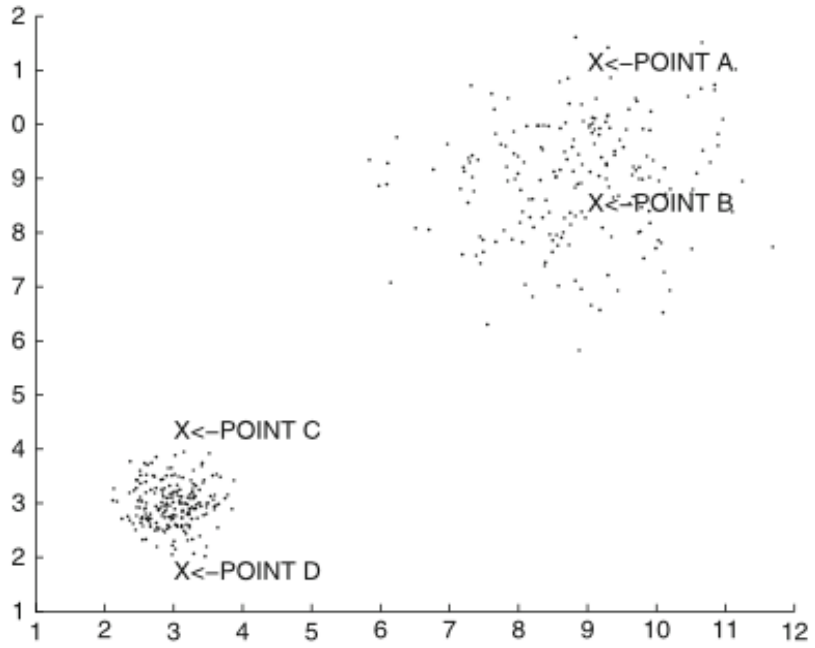
(a) A and C seem close (original data)



(b) A and C are actually far away (*ISOMAP* embedding)

Local data distribution

$$Dist(A, B) = Dist(C, D)?$$

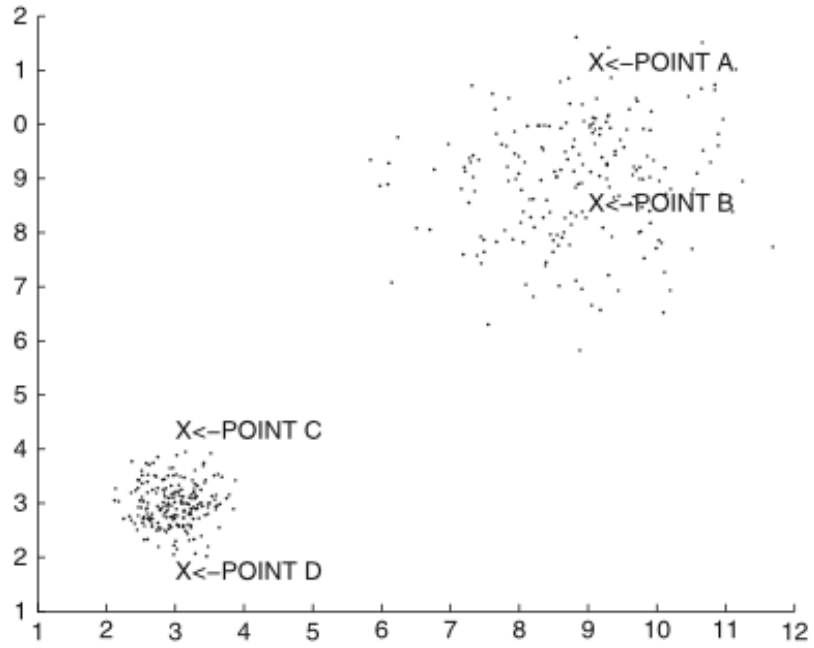


(a) local density variation

Local data distribution

$$Dist(A, B) = Dist(C, D)?$$

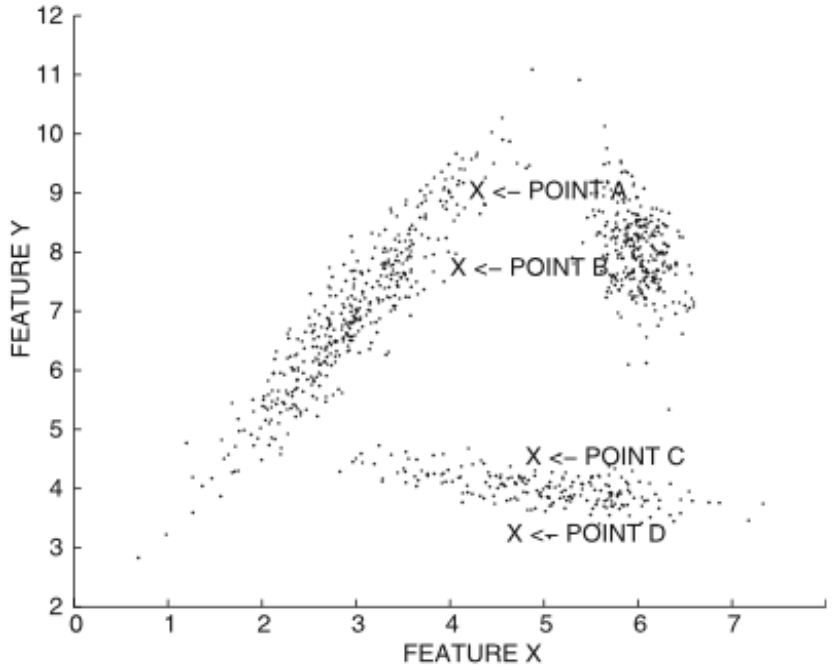
- The absolute distance is equal, but $Dist(C, D)$ should be considered greater due to the *local* data distribution
- We can consider a Shared Nearest Neighbor Similarity that is equal to the number of common neighbors between the two data points.



(a) local density variation

Local data distribution

$$Dist(A, B) = Dist(C, D)?$$



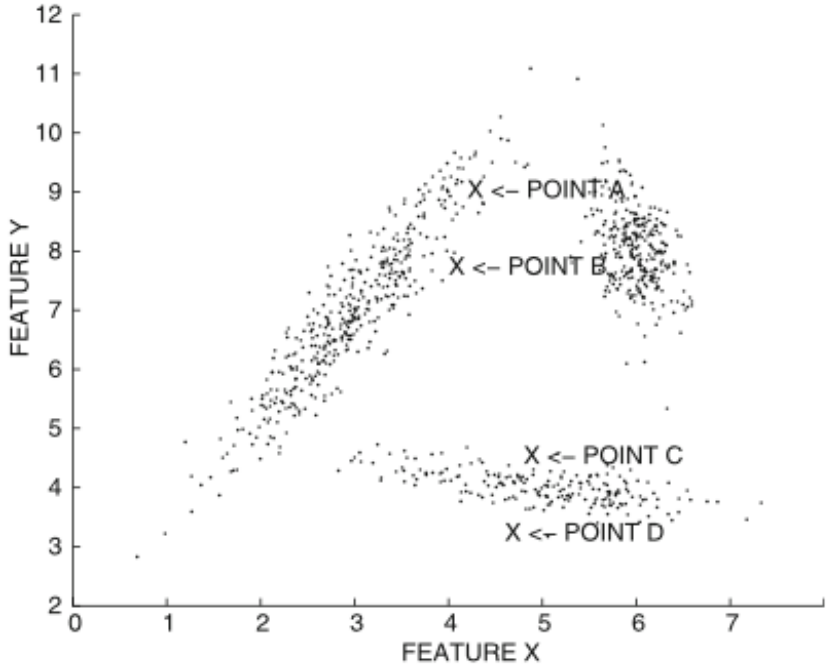
(b) local orientation variation

Local data distribution

$$Dist(A, B) = Dist(C, D)?$$

- The absolute distance is equal, the local clusters in each region show very different orientation.

- $Dist(C, D) \gg Dist(A, B)$

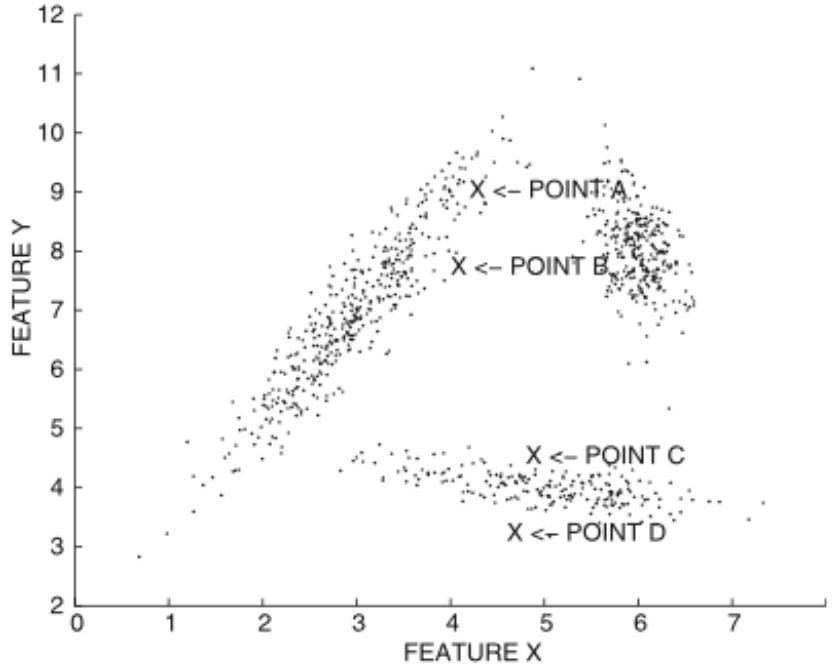


(b) local orientation variation

Local data distribution

$$Dist(A, B) = Dist(C, D)?$$

- We can partition the data into a set of local regions and calculate
- $Dist(A, B)$
  - Same region -> local statistics (Local Mahalanobis distance)
  - Different regions -> Global statistics or averaged statistics



(b) local orientation variation

Computational considerations

- Each of these methods have a cost
- Distance functions are executed repeatedly, whereas the preprocessing is performed only once.
- It is definitely advantageous to use a preprocessing-intensive approach to speed up later (distance or similarity) computations.

Categorical Data

- Data type portability? Binarization?
- Given two records $\bar{X} = (x_1 \dots x_d)$ and $\bar{Y} = (y_1 \dots y_d)$, the simplest possible similarity is the sum of the similarities on the individual attribute values.

$$Sim(\bar{X}, \bar{Y}) = \sum_{i=1}^{d} S(x_i, y_i)$$

- with the appropriate similarity function

$$S(x_i, y_i) = \begin{cases} 1 & \text{if } x_i = y_i \\ 0 & \text{otherwise} \end{cases}$$

Categorical Data

✅ Similarities are easy to consider compared with distances

❌ No data peculiarities are taken into account

- *data science* is equally not similar with *informatics* and *medicine*

❌ No relative frequency or statistical information are considered

Change the similarity function $S(x_i, y_i)$

Categorical Data

Inverse occurence frequency

- is a generalization of the similarity function $S(x_i, y_i)$
- leverages the similarity of the matching attributes with an inverse function of the frequency of the matched value.

$$S(x_i, y_i) = \begin{cases} 1/p_k(x_i)^2 & \text{if } x_i = y_i \\ 0 & \text{otherwise} \end{cases}$$

where $p_k(x)$ is the fraction of records in which the k-th attribute takes on the value of x in the data set.

Categorical Data

Goodall similarity

An higher similarity value is assigned to a match when the value is infrequent

$$S(x_i, y_i) = \begin{cases} 1 - p_k(x_i)^2 & \text{if } x_i = y_i \\ 0 & \text{otherwise} \end{cases}$$

where $p_k(x)$ is the fraction of records in which the k-th attribute takes on the value of x in the data set.

Mixed quantitative and categorical data

- Given two records $\bar{X} = (\overline{X_n}, \overline{X_c})$ and $\bar{Y} = (\overline{Y_n}, \overline{Y_c})$
  - $\overline{X_n}$ and $\overline{Y_n}$ are the subsets of numerical attributes
  - $\overline{X_c}$ and $\overline{Y_c}$ are the subsets of categorical attributes

- Weighted average

$$Sim(\bar{X}, \bar{Y}) = \lambda \cdot NumSim(\overline{X_n}, \overline{Y_n}) + (1 - \lambda) \cdot CatSim(\overline{X_c}, \overline{Y_c})$$

- Normalized weighted average

$$Sim(\bar{X}, \bar{Y}) = \lambda \cdot \frac{NumSim(\overline{X_n}, \overline{Y_n})}{\sigma_n} + (1 - \lambda) \cdot \frac{CatSim(\overline{X_c}, \overline{Y_c})}{\sigma_c}$$

Textual data

- When text is represented as a bag of words, thus a document is a vector of relative frequencies of the *d* words in its vocabulary, we usually have a very sparse vectors and a classic L_p-norm is inconsistent.

- Applying a dimensionality reduction via Latent Semantic Analysis, with meaningful vectors, L_p-norm can be applied.

Textual data

- Cosine similarity
  - Measures the angle between two documents that is insensitive to the absolute length of the documents.
  - Let $\bar{X} = (x_1 \dots x_d)$ and $\bar{Y} = (y_1 \dots y_d)$, be two documents on a lexicon of size d

$$\cos(\bar{X}, \bar{Y}) = \frac{\sum_{i=1}^{d} x_i \cdot y_i}{\sqrt{\sum_{i=1}^{d} x_i^2} \cdot \sqrt{\sum_{i=1}^{d} y_i^2}}$$

  - but we considered only the raw frequencies

Textual data

- Cosine similarity
  - A match over an uncommon word is more indicative of similarity than matches over frequent ones

$$\cos(\bar{X}, \bar{Y}) = \frac{\sum_{i=1}^{d} h(x_i) \cdot h(y_i)}{\sqrt{\sum_{i=1}^{d} h(x_i)^2} \cdot \sqrt{\sum_{i=1}^{d} h(y_i)^2}}$$

  - $h(\cdot)$ is the normalized frequency $h(x_i) = f(x_i) \cdot id_i$ or TF-IDF
  - $f(x_i)$ can be a square root or a logarithm, optionally applied to the frequencies before similarity computation.
  - $id_i = \log({}^{n}/_{n_i})$ is the inverse document frequency for *i*-th word

Binary and set data

- Jaccard coefficient
  - For binary multidimensional data representing a set-based data, where a value of 1 indicates the presence of an element in a set.
  - S_X and S_Y are two sets with binary representations $\bar{X}$ and $\bar{Y}$

$$J(\bar{X},\bar{Y}) = \frac{\sum_{i=1}^{d} x_i \cdot y_i}{\sum_{i=1}^{d} x_i^2 + \sum_{i=1}^{d} y_i^2 - \sum_{i=1}^{d} x_i \cdot y_i} = \frac{|S_X \cap S_Y|}{|S_X \cup S_Y|}$$

  - It is also referred as Intersection over Union (IoU)

Temporal series

- Time-Series Similarity Measures
  - L_p-norm
  - Dynamic Time Warping Distance (DTW)
  - Window based matching

- Discrete Sequence Similarity Measures
  - Edit distance (Levenshtein distance)
  - Longest common subsequence (LCSS)

Time series

- Euclidean distance? Yes, but
  - ❗Behavioral attribute scaling and translation
    - ✅ translation -> mean centering of behavioral attributes
    - ✅ scaling -> normalization
  - ❗Contextual attribute translation
    - Referring to the same interval
  - ❗Contextual attribute scaling
    - Referring to the same scale (time warping)
  - ❗Noncontiguity in matching
    - Noisy segments

L_p-norm

- Given two series $\bar{X} = (x_1 \dots x_d)$ and $\bar{Y} = (y_1 \dots y_d)$

$$Dist(\bar{X}, \bar{Y}) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

✅ Can be applied to raw series (considered as a multidimensional data point) or to their wavelet transformations

❌ Series must have equal length

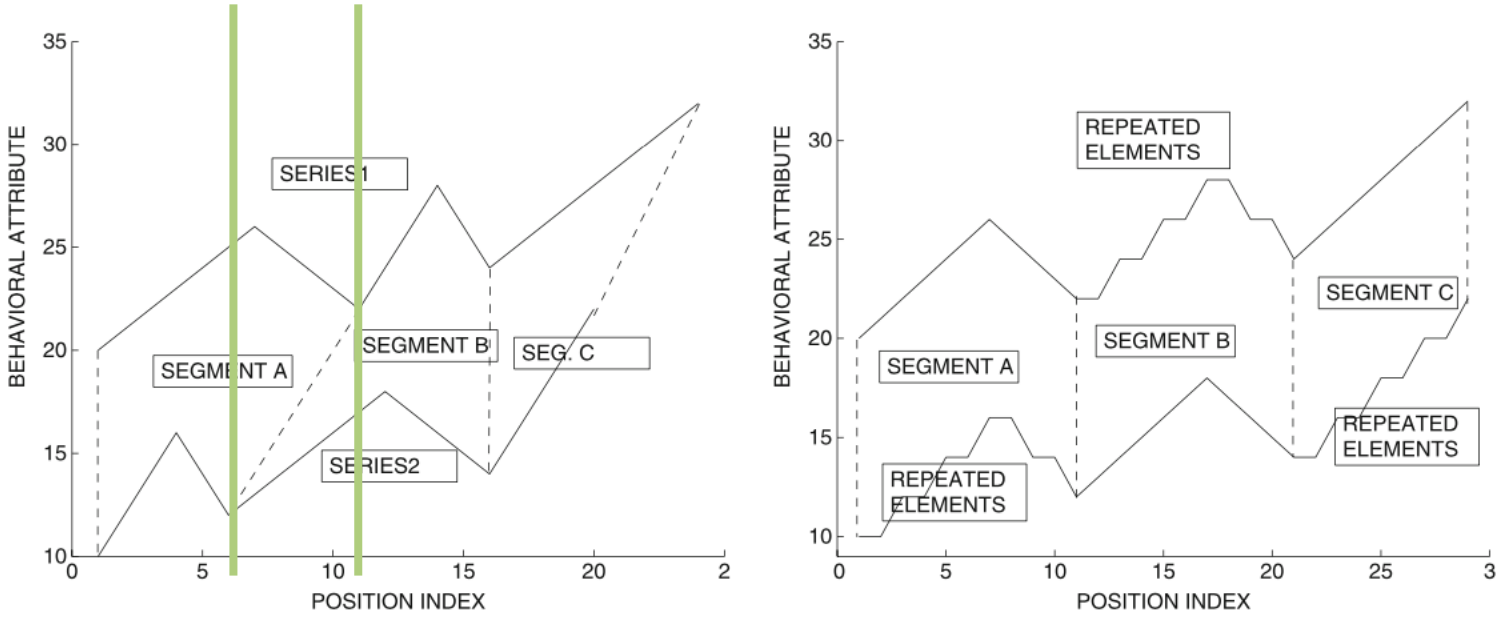
Dynamic Time Warping Distance (DTW)

- DTW stretches the series along the time axis dynamically over different portions to enable more effective matching.
- It addresses the issue of contextual attribute scaling.
- Can be applied to both time-series and sequence data.
- Series with different lengths can be measured, allowing a many to one mapping for time warping.

Dynamic Time Warping Distance (DTW)

- Many-to-one mapping can be thought of in terms of repeating some of the elements in carefully chosen segments of either of the two time-series

- On x axis we have the position index on the time series, on y axis the behavioral attribute and the green lines are the corresponde over the time values



(a) Original series (b) Warped series

Dynamic Time Warping Distance (DTW)

- Obtained series have the same length and can be measured via L_p-norm.
- The objective is to warp series in the optimal way, that is dynamically.
- Leveraging Manhattan distance *M*, the first *i*-th elements of two equal length series can be calculated recursively as

$$M(\overline{X}_i, \overline{Y}_i) = |x_i - y_i| + M(\overline{X_{i-1}}, \overline{Y_{i-1}})$$

Dynamic Time Warping Distance (DTW)

- Iterate recursively the process thus minimizing the new distance, i.e. not both series must be reduced.

$$DTW(i,j) = distance(x_i, y_i) + \min \begin{cases} DTW(i, j-1) & \text{repeat } x_i \\ DTW(i-1, j) & \text{repeat } y_i \\ DTW(i-1, j-1) & \text{repeat neither} \end{cases}$$

$$DTW(0,0) = 0, \qquad DTW(i,0) = DTW(0,j) = \infty$$

- Distance can be defined in a variety of ways.

Dynamic Time Warping Distance (DTW)

- Can be easily implemented via a double for cicle.
- Some constraints can be added such as a *window constraint* which DTW is calculated if $|i - j| \leq w$.
- The focus is on the contextual attributes, thus this method can be easily extended to multiple behavioral attributes.

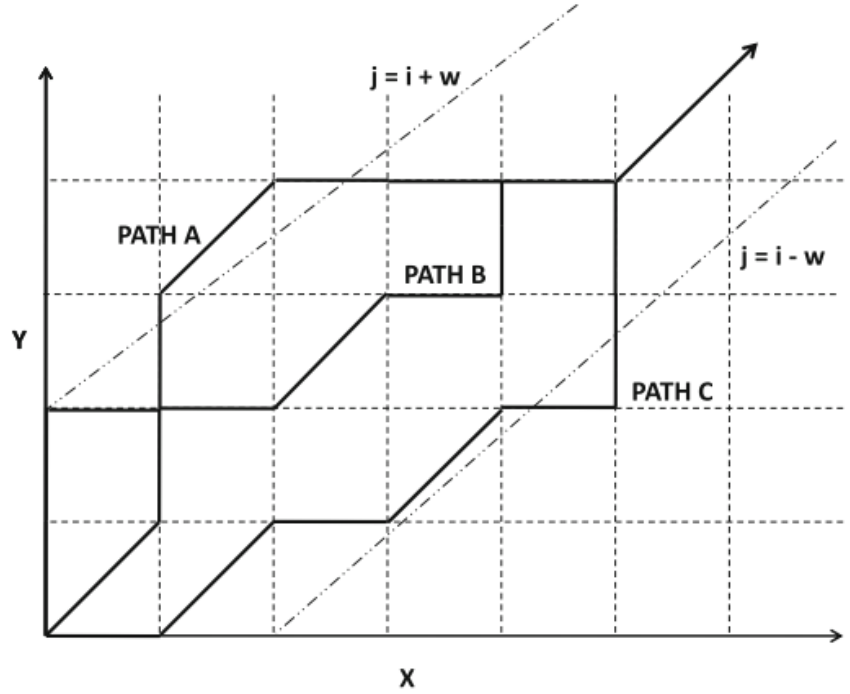


Figure 3.9: Illustration of warping paths

Window based matching

- Given two time series $\bar{X}$ and $\bar{Y}$, r nonoverlapping windows can be extracted from the respective series, thus a similarity can be calculated between each windows with an appropriate *Match* function

$$Sim(\bar{X}, \bar{Y}) = \sum_{i=1}^{r} Match(\bar{X}_i, \bar{Y}_i)$$

Discrete sequence

- General idea is the same as time series
- L_p-norm and DTW can be applied, but some peculiar methods for discrete sequences can be used:
  - Edit distance (Levenshtein distance)
  - Longest common subsequence (LCSS)

Edit distance (Levenshtein distance)

- Defined as the least amount of effort (or *cost*) required to transform one sequence into another with edits operations
  - symbol insertions, deletions, and replacements
  - $\text{cost}(\text{insert}) = \text{cost}(\text{delete})$
  - $\text{cost}(\text{replace}) = \text{cost}(\text{insert}) + \text{cost}(\text{delete})$

Edit distance (Levenshtein distance)

- It is an asymmetric distance
  - $Edit(\bar{X}, \bar{Y}) \neq Edit(\bar{Y}, \bar{X})\ if \text{cost(insert)} \neq \text{cost(delete)}$

$$abababab \text{ vs } babababa$$

Edit distance (Levenshtein distance)

- It is an asymmetric distance
  - $Edit(\bar{X}, \bar{Y}) \neq Edit(\bar{Y}, \bar{X})\ if \text{cost(insert)} \neq \text{cost(delete)}$

$$abababab \text{ vs } babababa$$

8 Replacements or 1 Deletion + 1 Insertion?
Consider the most optimal one!

Edit distance (Levenshtein distance)

$$Edit(i,j) = \min \begin{cases} Edit(i-1,j) + Deletion\ Cost \\ Edit(i,j-1) + Insertion\ Cost \\ Edit(i-1,j-1) + I_{ij} \cdot Replacement\ cost \end{cases}$$

$$I_{ij} = \begin{cases} 0 & x_i = y_i \\ 1 & \text{otherwise} \end{cases}; \quad \begin{matrix} Edit(i,0) = i\ deletions \\ Edit(0,j) = j\ insertions \end{matrix}$$

Longest common subsequence (LCSS)

- A subsequence of a sequence is a set of symbols drawn from the sequence in the same order as the original sequence and values of the subsequence not need not be contiguous.

- The longest the common subsequence over the two series, the most similar are the two sequences.

Longest common subsequence (LCSS)

- The longest the common subsequence over the two series, the most similar are the two sequences.

$$LCSS(i,j) = \max \begin{cases} LCSS(i-1,j-1)+1 & x_i = y_i \\ LCSS(i-1,j) & no\ match\ on\ x_i \\ LCSS(i,j-1) & no\ match\ on\ y_i \end{cases}$$

$$LCSS(i,0) = LCSS(0,j) = 0$$

Graphs

- Consider a graph $G(N, A)$ with N nodes and A edges
  - distance functions -> edges' costs c_{ij}
  - similarity functions -> edges' weights w_{ij}
- Homophily is the similarity between any pair of nodes, and nodes that are connected via short paths and many paths can be considered more similar.
- Methods
  - Shortest-path on the graph (Dijkstra algorithm)
  - Random Walk-Based Similarity

Dijkstra algorithm

- Objective is to measure the distances from source node *s* to any other node in the network

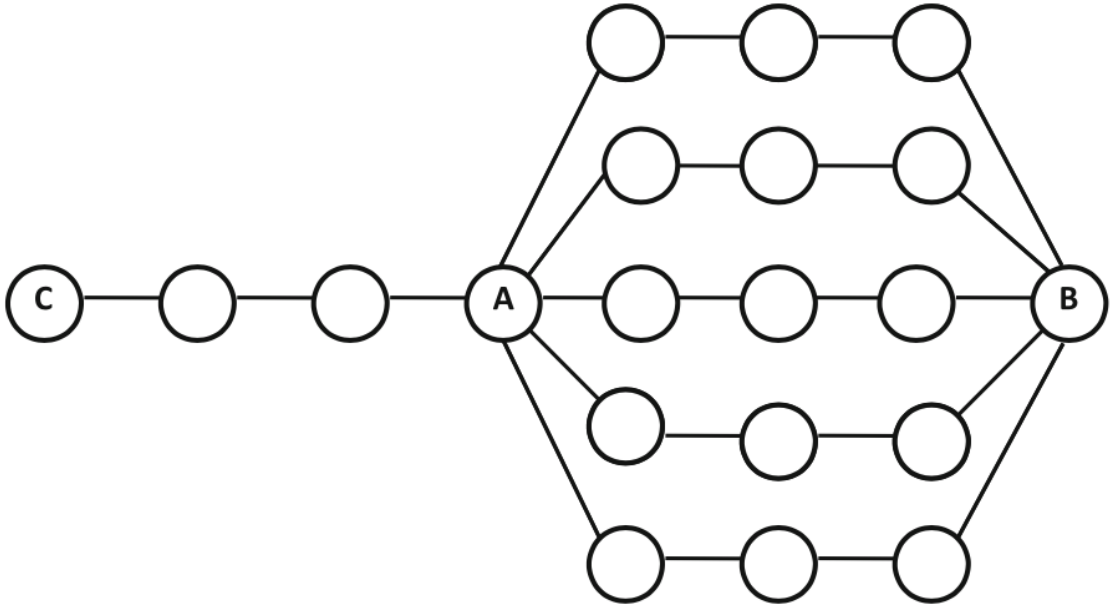
$$SP(s,j) = \min\{SP(s,j), SP(s,i) + c_{ij}\}$$

$$SP(s,s) = 0;\ SP(s,j) = \infty\ \forall\ s \neq i$$

- This approach is linear in the number of edges in the network, since each node is examined once.
- Obtanied values relies on structural distances, and do not leverage the multiplicity in paths between a pair of nodes because the focus is only on the raw structural distances.

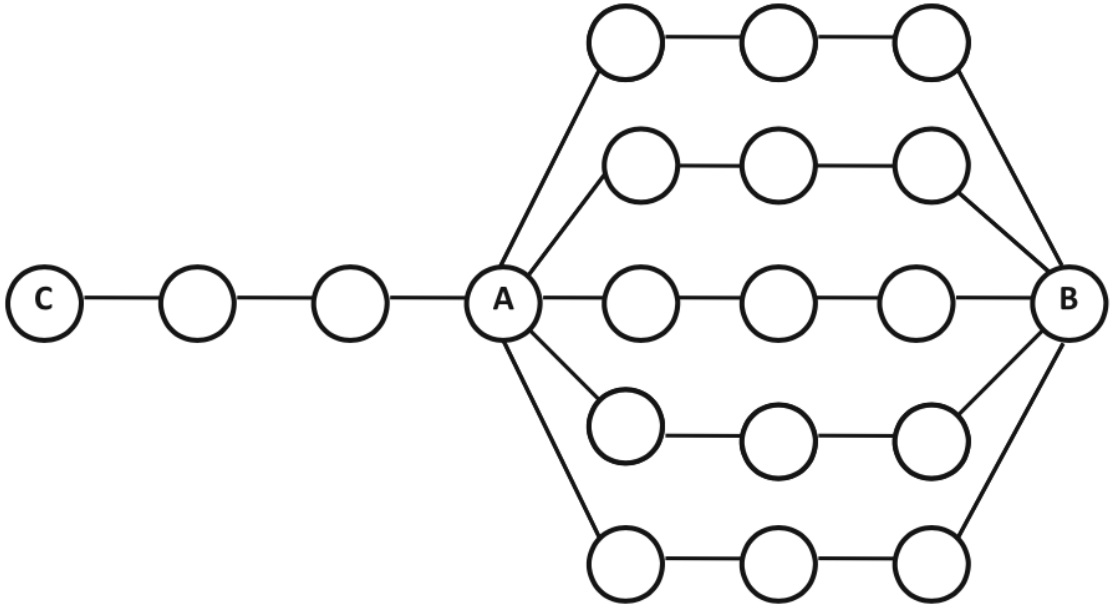
Random Walk-Based Similarity

$Sim(A, C) > Sim(A, B)$ or $Sim(A, B) > Sim(A, C)$?



Random Walk-Based Similarity

$Sim(A, C) > Sim(A, B)$ or $Sim(A, B) > Sim(A, C)$?



There is a multiplicity of paths connecting A and B, they should be considered more similar

Random Walk-Based Similarity

- The idea is to consider two nodes similar if are tightly connected with a multiplicity of paths, despite two nodes connected by a shortest path.
  - Starting form node S, an adjacent node is visited, with a weighted probability proportional to w_{ij}
  - Each node has a "jump back" to the source node S with a *restart probability*, thus it will result in a probability distribution heavily biased towards S.
  - Nodes that are more similar to S will have higher probability of visits.

Random Walk-Based Similarity

- Similarity is obtained maximizing this probability, since most probable paths connect *S* to its similar nodes in the graph.

If you were lost in a road network and drove randomly, which location are you more likely to reach?

A close location that can be reached in multiple ways.

Graphs

- What if I have multiple graphs?
- Isomorphism is the similarity between two graphs, and it is a NP-hard problem (nondeterministic polynomial time)
  - Maximum common subgraph distance
    - When two graphs contain a large subgraph in common, they can be considered more similar.
  - Substructure-based similarity
    - Similarity measure obtained counting the frequently occurring substructures between the two graphs.
  - Graph-edit distance
    - Number of edits required to transform one graph to the other.